DSA MASTERY COMPLEXITY HANDBOOK

Complexity Handbook

Time and space complexity, explained so you never forget

Seven everyday scenarios, one growth chart, diagrams for every class, best, worst and amortized cases, memory cost, a quiz, and code in six languages

$O(1)$
Same work at any size

$O(n^2)$
Doubling the input quadruples the work

$O(2^n)$
Each new item doubles the work

Time ladder 4
Best / worst 9
Amortized 10
Calculate it 10
Space 12
Quiz 21

WHAT YOU WILL LEARN

- What complexity really measures, and why it is not seconds.
- The seven time-complexity classes, each tied to a real-world scenario you can picture.
- Best, average and worst case, plus Big-O, Big-Omega and Big-Theta.
- Amortized cost, and how to calculate complexity from code.
- Space (memory) complexity, the call stack, and the time-space trade-off.
- A one-page memory story and a quiz to lock it all in.

Contents

1 The Big Idea . . . 2
2 All Growth Curves in One Picture . . . 3
3 The Time Complexity Ladder . . . 4
4 Feel the Numbers . . . 8
5 Best, Average and Worst Case . . . 9
6 Amortized Cost . . . 10
7 How to Calculate Complexity . . . 10
8 Space (Memory) Complexity . . . 12
9 The Time-Space Trade-off . . . 13
10 Cheat Sheets . . . 14
11 The Code, in Six Languages . . . 14
12 Never-Forget Page and Quiz . . . 21

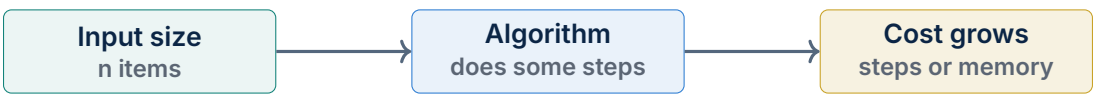
1 The Big Idea

Complexity answers one question: **when the input gets bigger, how much does the work (or memory) grow?**

SCENARIO: THE PIZZA DELIVERY

You deliver pizzas. Delivering **1** pizza to one house takes a while. The real question is what happens when you must deliver to **1,000** houses. If each house costs the same, the work grows in a straight line. If every house also needs a handshake with every other customer, the work explodes. *Complexity is the shape of that growth, not the stopwatch time.*

Why not just measure seconds? Seconds depend on the computer, the language and what else is running. The *shape* of growth does not change: a slow laptop and a fast server both take about four times longer when an **O(n^2)** algorithm gets double the data.



Big-O describes how the right box grows as the left box grows

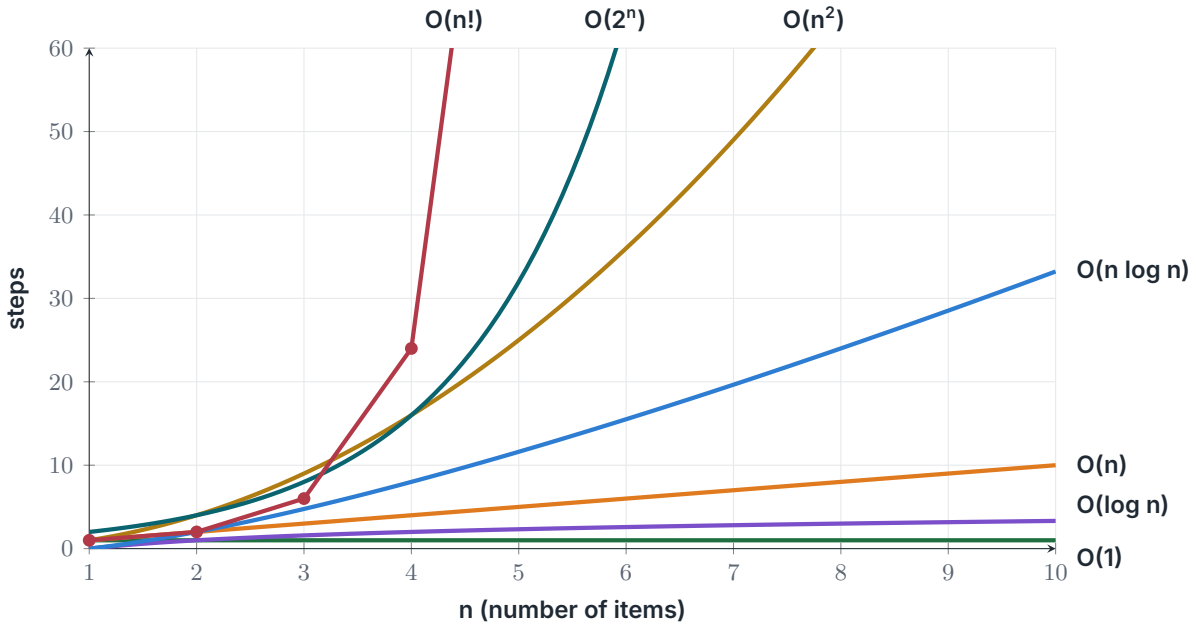
Four rules of Big-O

1. **Look at the worst growth.** Describe what happens as n becomes very large.
2. **Drop constants.** $3n$ and $100n$ are both **O(n)**.
3. **Drop smaller terms.** $n^2 + 50n + 7$ is **O(n^2)**; when n is huge, n^2 dominates.
4. **By default, quote the worst case** unless the question names another case.

Exact steps	Why	Big-O
5	constant	**O(1)**
$3n + 4$	drop 4, drop 3	**O(n)**
$n^2 + 50n + 7$	n^2 dominates	**O(n^2)**
$2n \log n + n$	$n \log n$ dominates n	**O(n log n)**

2 All Growth Curves in One Picture

Seven curves, one chart. The number of steps (vertical) as the input size n (horizontal) grows from 1 to 10. Three of the curves shoot off the top of the chart before n even reaches 8. That is what "explosive" looks like.



HOW TO READ THE CHART

Curves that hug the bottom (**O(1)**, **O(log n)**, **O(n)**) stay cheap as data grows. **O(n log n)** bends up gently. **O(n^2)** is already climbing steeply, while **O(2^n)** and **O(n!)** leave the chart almost immediately. The curves for **O(2^n)** and **O(n!)** cross 60 steps at about $n = 6$ and $n = 5$. At $n = 20$ they are in the millions and quintillions.

The Party Day story: never forget the order

Memorise the order with one story. A single day, seven scenes, from calm to chaos:

O(1)
1. Flick the light switch
One action, no matter how big the building.

O(log n)
2. Open the dictionary
Open the middle, throw away half. Repeat.

O(n)
3. Scan the bookshelf
Check every book once.

O(n log n)
4. Sort a deck of cards
Split in halves, then merge back.

$O(n^2)$
5. Shake everyone's hand
Every person meets every other person.

$O(2^n)$
6. Order every pizza combo
Each topping: in or out.

O(n!)
7. Argue about seating
Every possible order of the guests.

Green is safe, yellow is fine, orange is careful, red is danger.

REMEMBER

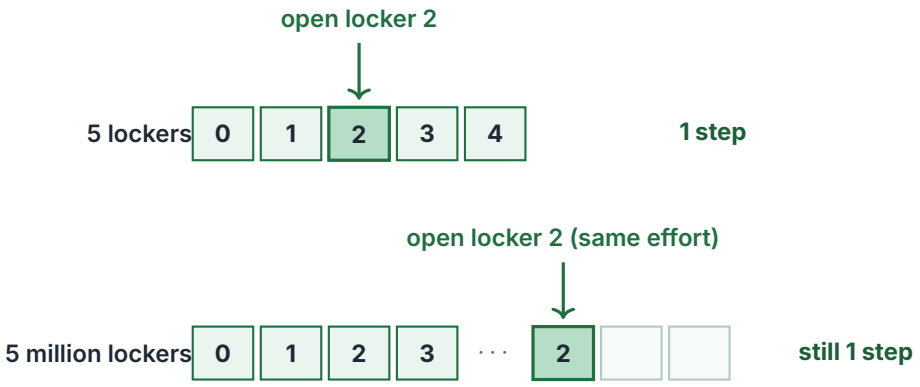
"I **flick** the light, **open** the dictionary, **scan** the shelf, **sort** the deck, **shake** every hand, **order** every pizza, then **argue** about seating."

3 The Time Complexity Ladder

Each rung gets a scenario, a diagram and a feel for how it scales. The code for every rung (in six languages) is in section 11.

Constant time

O(1) Constant
Same effort whether there are 5 items or 5 million. Elevator button, light switch, locker by number.

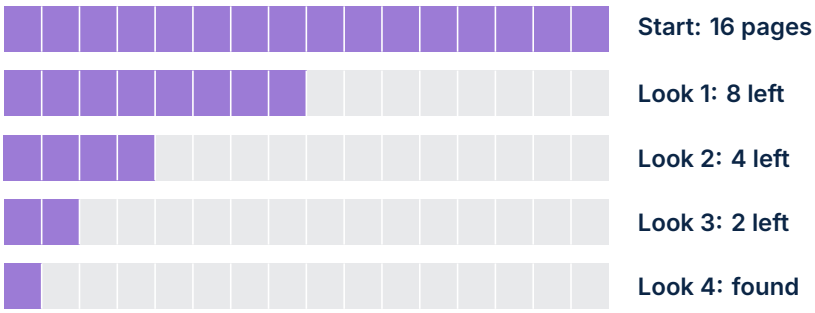


Why: an array finds locker i with one address calculation (see Lesson 1). **Examples:** array access by index, push onto a stack, looking up a key in a hash table (on average).

Logarithmic time

O(log n) Logarithmic

Every step throws away half. Looking up a word in a dictionary, guessing a number with “higher or lower”.



The trick: the data must be *sorted*. Open the middle page, decide which half holds your word, discard the other half, repeat.

Items		Halvings needed
16	→	4
1,000,000	→	at most 20
1,000,000,000	→	at most 30

A billion items in about 30 looks. That is why **O(log n)** feels like magic. **Examples:** binary search, lookups in a balanced search tree.

REMEMBER

Log means “how many times can I halve n before I reach 1?”

Linear time

O(n) Linear

Touch every item once. Hunting for a book on an unsorted shelf.

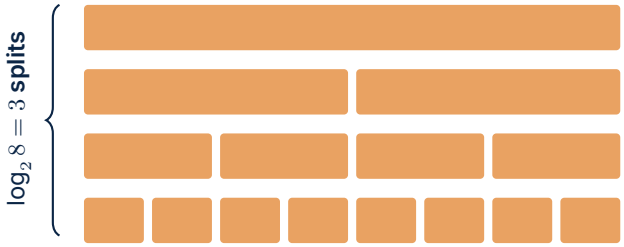


If the book is last (or missing) you look at all n books. **Double the books, double the work.** **Examples:** linear search, finding the maximum, summing a list, printing every item.

Linearithmic time

O(n log n) Linearithmic

Split into halves ($\log n$ levels), and do n work on every level. Sorting a deck of cards by merging.



Level 0: 8 items

Level 1: 4 + 4

Level 2: 2 + 2 + 2 + 2

Level 3: 1 each

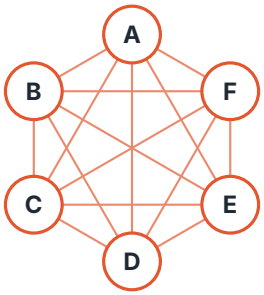
every level handles all 8 items

Total work = (number of levels) $\times$ (work per level) = $\log n \times n$. **Examples:** merge sort, heap sort, quick sort on average. A known fact: no sorting algorithm based on comparing items can beat **O(n log n)** in the worst case.

Quadratic time

O(n^2) Quadratic

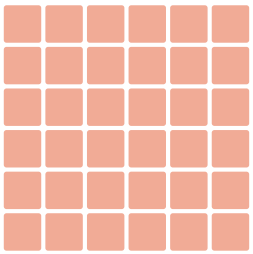
Every item meets every other item. A party where everyone shakes everyone's hand.



6 people, 15 handshakes

People n	Handshakes n(n−1)/2
2	1
5	10
6	15
10	45
100	4,950
1,000	499,500

The count is $n(n-1)/2$, which behaves like $n^2/2$, and the constant is dropped: **O(n^2)**. **Double the people, four times the handshakes.** In code it shows up as a loop inside a loop:



```
for each of n items
  for each of n items
    do one step
```

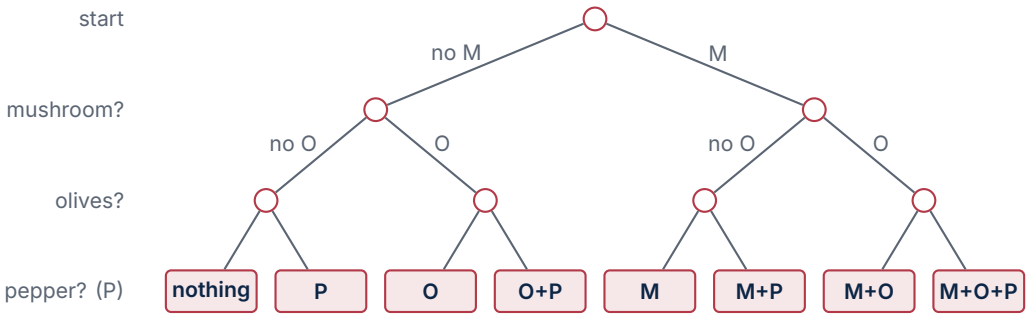
6 $\times$ 6 = 36 steps, n $\times$ n

Examples: bubble sort, selection sort, comparing every pair, naive two-sum. A triple-nested loop (naive multiplication of two $n \times n$ matrices) gives **O(n^3)**.

Exponential time

$O(2^n)$ Exponential

Each extra item doubles the work. Choosing which pizza toppings to include.



3 toppings give $2^3 = 8$ pizzas (including the empty one). Add a 4th topping and you get 16. **Every new item doubles the work.**

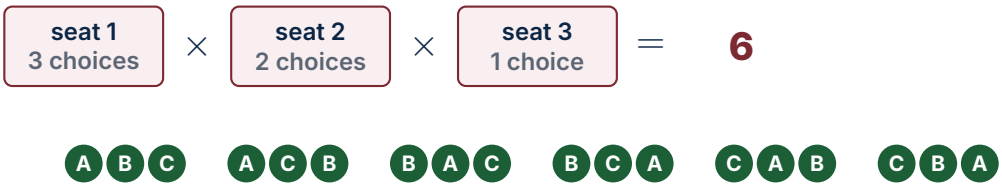
Toppings	10	20	30	50
Combinations	1,024	1,048,576	about 1.07 billion	about 1.1×10^{15}

Examples: generating all subsets, brute-force password guessing, and the naive recursive Fibonacci (which is exponential, about 1.6^n calls, commonly bounded as **$O(2^n)$**).

Factorial time

$O(n!)$ Factorial

Try every possible ordering. Seating guests, or a salesperson visiting every city in every order.



For n guests there are $n \times (n-1) \times \cdots \times 1 = n!$ orderings.

Guests	3	5	10	20
Orderings	6	120	3,628,800	about 2.4×10^{18}

Examples: brute-force travelling salesman, generating all permutations. In practice, **$O(n!)$** is only usable for n up to about 10 to 12.

4 Feel the Numbers

Steps taken at different input sizes

Class	n = 10	n = 20	n = 50	n = 100
O(1)	1	1	1	1
O(log n)	3.3	4.3	5.6	6.6
O(n)	10	20	50	100
O(n log n)	33	86	282	664
$O(n^2)$	100	400	2,500	10,000
$O(2^n)$	1,024	1,048,576	1.1×10^{15}	1.3×10^{30}
O(n!)	3,628,800	2.4×10^{18}	3.0×10^{64}	9.3×10^{157}

Values are rounded; log is base 2.

What that means on a real clock

Assume a computer does about **one billion simple steps per second** (1 step = 1 nanosecond). That is a rough planning figure; real machines vary.

Class	Input	Time, at 1 step = 1 ns
O(log n)	1 million items	about 20 steps, **about 20 nanoseconds**
O(n)	1 million items	1 million steps, **about 1 millisecond**
O(n log n)	1 million items	about 20 million steps, **about 0.02 seconds**
$O(n^2)$	1 million items	10^{12} steps, **about 17 minutes**
$O(2^n)$	50 items	1.1×10^{15} steps, **about 13 days**
O(n!)	20 items	2.4×10^{18} steps, **about 77 years**

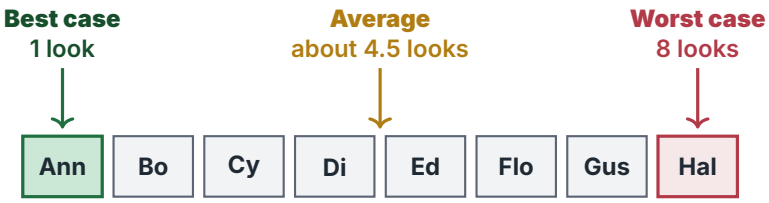
THE LESSON

The same million items take **0.02 seconds** with an **O(n log n)** sort and **17 minutes** with an **$O(n^2)$** one. Choosing the right algorithm matters more than buying a faster computer. And **$O(2^n)$** and **O(n!)** cannot be rescued by hardware at all.

5 Best, Average and Worst Case

The same algorithm can be lucky or unlucky depending on the input.

> **SCENARIO: FINDING A FRIEND IN YOUR CONTACT LIST**
> Your list is unsorted and you scan from the top. If your friend is first, you are done in 1 look. If they are last (or not in the list), you check everyone.



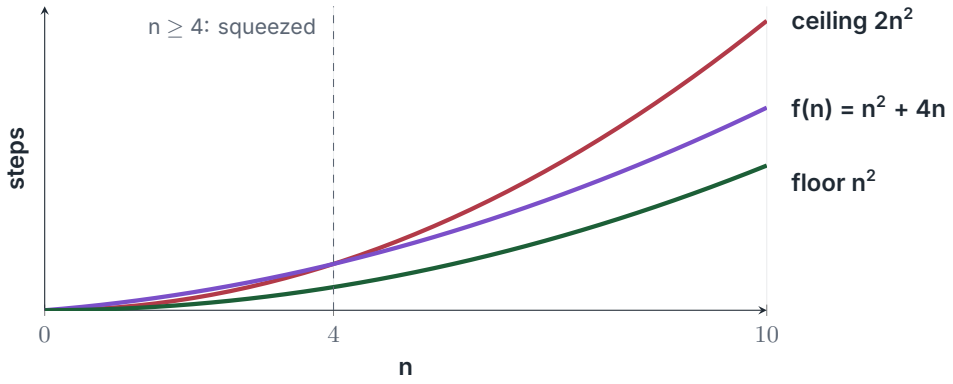
Case	Meaning	Linear search
Best	The luckiest input	**O(1)**
Average	A typical input (average over all inputs)	**O(n)** (about $n/2$ looks)
Worst	The unluckiest input	**O(n)** (n looks)

When we say an algorithm is "**O(n)**" with no other words, we mean the **worst case**. It is a promise: it will never be slower than this growth.

Big-O, Big-Omega and Big-Theta

Three symbols describe the same growth from different sides. Think of your commute to work:

Symbol	Reads as	Meaning	Commute analogy
O (Big-O)	"at most"	Upper bound, a ceiling	"It never takes more than 1 hour."
Ω (Big-Omega)	"at least"	Lower bound, a floor	"It never takes less than 20 minutes."
Θ (Big-Theta)	"exactly this growth"	Both bounds at once	"It always takes about 30 to 40 minutes."



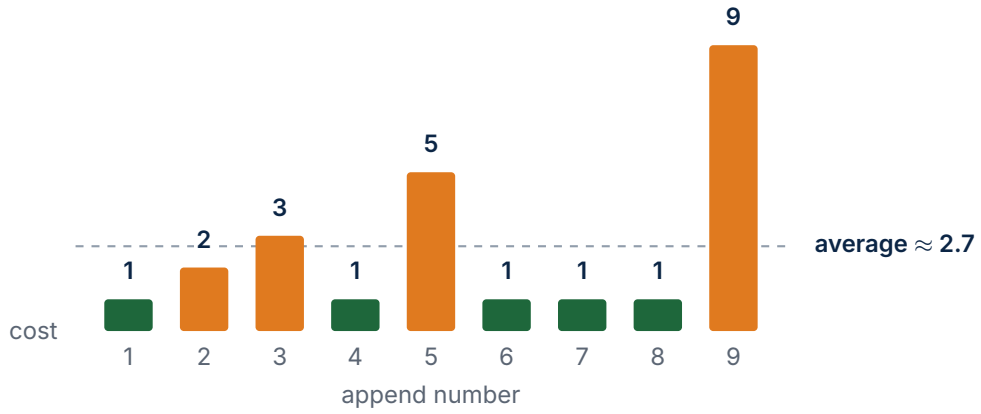
For $f(n) = n^2 + 4n$: once $n \geq 4$ it is squeezed between n^2 (floor) and $2n^2$ (ceiling), so it is $\Theta(n^2)$, which also means it is **O(n²)** and $\Omega(n^2)$. In everyday interviews people use "Big-O" to mean this tight worst-case growth.

6 Amortized Cost

SCENARIO: MONTHLY RENT

Rent is expensive on the first of the month and free on every other day. Averaged over the month, living costs a small, steady amount per day. *Amortized* cost is that average: spread an occasional expensive step across all the cheap ones.

The classic case is appending to a **dynamic array** (Python list, Java `ArrayList`, Go slice, Rust `Vec`). When full, it allocates a block twice as big, copies everything across, then continues.



Cost model: one write per append, plus one copy for every item moved when the array grows (capacity 1, 2, 4, 8, 16, ...). Appends 2, 3, 5 and 9 trigger a resize; the other five are a single write. Total for these 9 appends is $1+2+3+1+5+1+1+1+9=24$, which is about 2.7 per append.

Why it stays small forever: resizes happen at sizes 1, 2, 4, 8, ..., so all copying together is $1+2+4+\cdots<2n$. Add the n plain writes and the total is under $3n$, so **each append costs about 3 steps on average: O(1) amortized.**

REMEMBER

Amortized is not "average over random inputs". It is the average over a *sequence of operations*, with a guarantee for the whole sequence.

7 How to Calculate Complexity

Five questions, in order

Ask these of any piece of code and the first "yes" usually names the class. (Halve-then-merge, meaning n items across $\log n$ levels, is the one combination to remember: **O(n log n)**.)

1. Do I just do a fixed number of steps, with no loop over the input?	O(1)
2. Does the problem *shrink by half* every step?	O(log n)
3. Do I touch each item *once*?	O(n)
4. Do I touch every *pair* (a loop inside a loop)?	$O(n^2)$
5. Do I try every *subset* (in or out) or every *ordering*?	$O(2^n)$ or O(n!)

Rules for combining

Pattern	Rule	Result
Step, then another step	**Add**	**O(n) + O(n) = O(n)**
Loop inside a loop	**Multiply**	**O(n) $\times$ O(n) = O(n^2)**
Constant factors	**Drop** them	$10n \to$ **O(n)**
Smaller terms	**Drop** them	$n^2 + n \to$ **O(n^2)**
Two different inputs	**Use different letters**	Loop over a inside loop over $b \to$ **O(a $\times$ b)**
Halving or doubling the counter	Count how many times	**O(log n)**

Six snippets, analysed

Python: complexity of six small functions

```python
def a(arr):                    # one loop                        -> O(n)
    total = 0
    for x in arr:
        total += x
    return total

def b(arr):                    # two loops, one after the other -> O(n + n) = O(n)
    total = 0
    for x in arr:
        total += x
    for x in arr:
        total -= x
    return total

def c(arr):                    # loop inside a loop              -> O(n * n) = O(n^2)
    pairs = 0
    for x in arr:
        for y in arr:
            pairs += 1
    return pairs

def d(arr):                    # constant inner loop             -> O(10n) = O(n)
    steps = 0
    for x in arr:
        for _ in range(10):
            steps += 1
    return steps

def e(n):                      # doubling until n                -> O(log n)
    i, steps = 1, 0
    while i < n:
        i *= 2
        steps += 1
    return steps

def f(a_list, b_list):         # two different inputs            -> O(a * b)
    steps = 0
    for x in a_list:
        for y in b_list:
            steps += 1
    return steps

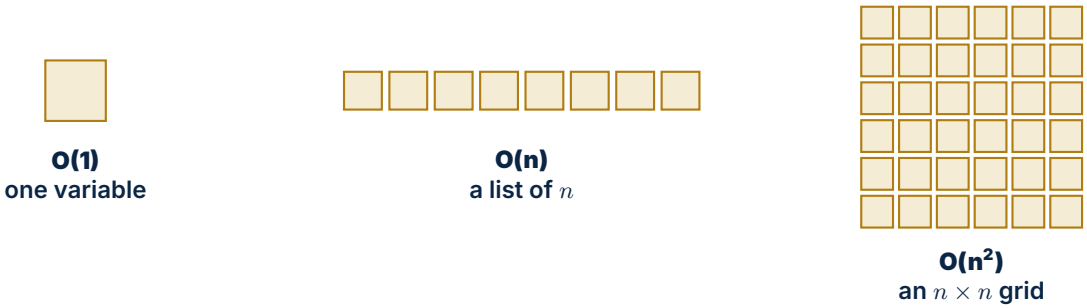
print(a([1, 2, 3]), b([1, 2, 3]), c([1, 2, 3]), d([1, 2, 3]), e(16), f([1, 2], [1, 2, 3]))
```

8 Space (Memory) Complexity

Space complexity measures the *extra memory* an algorithm needs as the input grows. By convention it counts the **auxiliary space** (extra memory the algorithm itself allocates). Sometimes the input itself is counted too; always check which one is meant.

SCENARIO: COOKING DINNER

- **One cutting board, reused for everything** → **O(1)** space. The board never multiplies, however many ingredients you cook.
- **A separate bowl for each of n ingredients** → **O(n)** space.
- **A tasting plate for every pair of ingredients** → **O(n^2)** space.

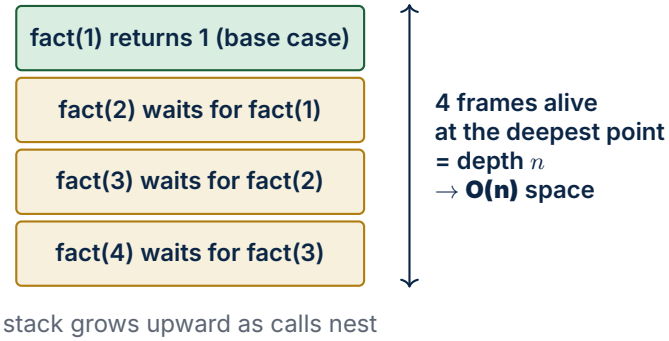


The kinds of space cost

Space	Typical cause	Example
O(1)	A few variables; work done *in place*	Reversing an array with two pointers
O(log n)	Recursion that halves the problem	Recursive binary search (stack depth)
O(n)	A copy, a hash set, or n-deep recursion	Hash map in two-sum; merge sort buffer
O(n^2)	A 2-D table	Adjacency matrix, DP grid

The hidden cost: the call stack

Every function call that is still waiting for an answer keeps a **frame** on the call stack. A recursion that goes n deep holds n frames at once, so it uses **O(n)** space even if it allocates nothing else.



stack grows upward as calls nest

STACK OVERFLOW

The stack is limited. In Python the default limit is about 1,000 nested calls (`sys.getrecursionlimit()`), and going deeper raises `RecursionError`. In Java and others the limit depends on the thread's stack size. This is why very deep recursion is rewritten as a loop (**O(1)** space).

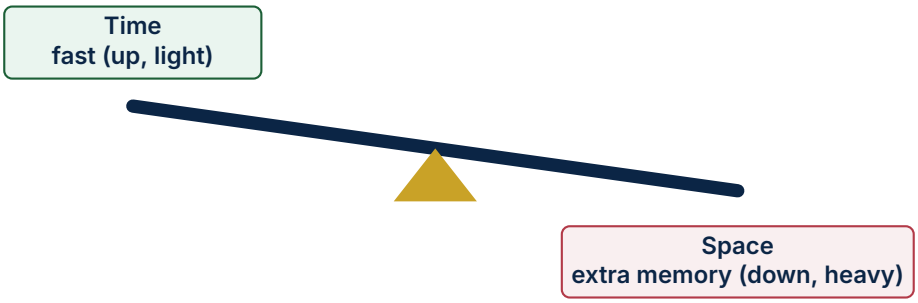
Why $O(n^2)$ space hurts: a real number

A social network with 100,000 users stored as an **adjacency matrix** (one cell for every pair of users) needs $100{,}000^2 = 10^{10}$ cells. At one byte per cell that is about **10 gigabytes**; packed as single bits it is about 1.25 GB. A list of each user's friends only grows with the actual friendships, which is far smaller for a typical network. Space complexity decides whether the program fits in memory at all.

9 The Time-Space Trade-off

SCENARIO: MEAL PREP

Cooking fresh every night uses *no* fridge space but costs time every day. Meal-prepping on Sunday uses *fridge space* but makes each dinner instant. Extra memory buys speed. Algorithms often let you choose.



Example: two-sum (find two numbers that add to a target).

Approach	Idea	Time	Space
Brute force	Try every pair	**$O(n^2)$**	**O(1)**
Hash map	Remember numbers seen; look up the partner	**O(n)**	**O(n)**

Python: two-sum, brute force vs hash map

```python
# Problem: find two numbers in `nums` that add up to `target`.

def two_sum_brute(nums, target):          # Time O(n^2), Space O(1)
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return (i, j)
    return None

def two_sum_hash(nums, target):           # Time O(n), Space O(n)
    seen = {}                             # value -> index (the extra memory)
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i
    return None

print(two_sum_brute([3, 8, 2, 5], 10))    # (1, 2)
print(two_sum_hash([3, 8, 2, 5], 10))     # (1, 2)
```

Another trade: *memoization*. The naive recursive Fibonacci recomputes the same values over and over (exponential time). Storing each answer the first time it is computed brings the time down to **O(n)** at the cost of **O(n)** extra memory.

10 Cheat Sheets

Common data structures

Structure	Access	Search	Insert	Delete
Array	**O(1)**	**O(n)**	**O(n)** (end: **O(1)** amortized)	**O(n)**
Linked list	**O(n)**	**O(n)**	**O(1)** at head	**O(1)** at head
Stack / queue	–	**O(n)**	**O(1)**	**O(1)**
Hash table	–	**O(1)** avg, **O(n)** worst	**O(1)** avg	**O(1)** avg
Balanced search tree	–	**O(log n)**	**O(log n)**	**O(log n)**
Binary heap	min/max: **O(1)**	**O(n)**	**O(log n)**	**O(log n)** (min/max)

Hash table worst case happens when many keys collide. A plain (unbalanced) search tree can degrade to **O(n)** if it becomes a long chain. We will study each structure in its own lesson.

Common sorting algorithms

Algorithm	Best	Average	Worst	Extra space
Bubble sort (with early exit)	**O(n)**	**O(n^2)**	**O(n^2)**	**O(1)**
Selection sort	**O(n^2)**	**O(n^2)**	**O(n^2)**	**O(1)**
Insertion sort	**O(n)**	**O(n^2)**	**O(n^2)**	**O(1)**
Merge sort	**O(n log n)**	**O(n log n)**	**O(n log n)**	**O(n)**
Quick sort	**O(n log n)**	**O(n log n)**	**O(n^2)**	**O(log n)** (recursion stack)
Heap sort	**O(n log n)**	**O(n log n)**	**O(n log n)**	**O(1)**

Quick sort's worst case needs a very unlucky pivot choice on every step, which is why it is fast in practice. Binary search: **O(log n)** time, **O(1)** space (loop) or **O(log n)** space (recursive).

11 The Code, in Six Languages

One program per language counts the steps each class really takes, using $n = 16$ for the polynomial classes, $n = 10$ for **O(2^n)** and $n = 5$ for **O(n!)**. It also measures three kinds of space. All six programs were compiled and run and print the same numbers:

```
O(1)                    1
O(log n)                4
O(n)                   16
O(n log n)             64
O(n^2)                256
O(2^n)               1024
O(n!)                 120
space O(n) copy        16
space O(n^2) grid     256
space O(n) recursion   16
```

The recursion line counts the frames alive for `depth(15)`: 16 frames, one per value 15 down to 0. (TypeScript needs `tsc --target es2020`.)

Python

Python 3

```python
def constant(a):                # O(1): one step, whatever the size
    _ = a[0]
    return 1

def logarithmic(n):             # O(log n): halve until 1
    steps = 0
    while n > 1:
        n //= 2
        steps += 1
    return steps

def linear(n):                  # O(n): touch each item once
    steps = 0
    for _ in range(n):
        steps += 1
    return steps

def linearithmic(n):            # O(n log n): n items, each halved log n times
    steps = 0
    for _ in range(n):
        m = n
        while m > 1:
            m //= 2
            steps += 1
    return steps

def quadratic(n):               # O(n^2): every pair
    steps = 0
    for _ in range(n):
        for _ in range(n):
            steps += 1
    return steps

def exponential(n):             # O(2^n): each item is in or out
    if n == 0:
        return 1
    return exponential(n - 1) + exponential(n - 1)

def factorial_time(n):          # O(n!): every ordering
    if n == 0:
        return 1
    return sum(factorial_time(n - 1) for _ in range(n))

# ---- space ----
def copy_space(n):              # O(n) extra memory
    return len(list(range(n)))

def grid_space(n):              # O(n^2) extra memory
    return sum(len([0] * n) for _ in range(n))

def depth(n):                   # O(n) call stack
    return 1 if n == 0 else 1 + depth(n - 1)

print("O(1)      ", constant([5, 6, 7]))
print("O(log n)  ", logarithmic(16))
print("O(n)      ", linear(16))
print("O(n log n)", linearithmic(16))
print("O(n^2)    ", quadratic(16))
print("O(2^n)    ", exponential(10))
print("O(n!)     ", factorial_time(5))
print("space O(n) copy      ", copy_space(16))
print("space O(n^2) grid    ", grid_space(16))
print("space O(n) recursion ", depth(15))
```

Java

Java

```java
public class ComplexityDemo {
    static int constant(int[] a) { int x = a[0]; return 1; }          // O(1)

    static int logarithmic(int n) {                                    // O(log n)
        int steps = 0;
        while (n > 1) { n /= 2; steps++; }
        return steps;
    }

    static int linear(int n) {                                         // O(n)
        int steps = 0;
        for (int i = 0; i < n; i++) steps++;
        return steps;
    }

    static int linearithmic(int n) {                                   // O(n log n)
        int steps = 0;
        for (int i = 0; i < n; i++)
            for (int m = n; m > 1; m /= 2) steps++;
        return steps;
    }

    static int quadratic(int n) {                                      // O(n^2)
        int steps = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) steps++;
        return steps;
    }

    static int exponential(int n) {                                    // O(2^n)
        if (n == 0) return 1;
        return exponential(n - 1) + exponential(n - 1);
    }

    static int factorialTime(int n) {                                  // O(n!)
        if (n == 0) return 1;
        int total = 0;
        for (int i = 0; i < n; i++) total += factorialTime(n - 1);
        return total;
    }

    // ---- space ----
    static int copySpace(int n) { return new int[n].length; }          // O(n)
    static int gridSpace(int n) { return new int[n][n].length * n; }   // O(n^2)
    static int depth(int n) { return n == 0 ? 1 : 1 + depth(n - 1); }  // O(n) stack

    public static void main(String[] args) {
        System.out.println("O(1)       " + constant(new int[]{5, 6, 7}));
        System.out.println("O(log n)   " + logarithmic(16));
        System.out.println("O(n)       " + linear(16));
        System.out.println("O(n log n) " + linearithmic(16));
        System.out.println("O(n^2)     " + quadratic(16));
        System.out.println("O(2^n)     " + exponential(10));
        System.out.println("O(n!)      " + factorialTime(5));
        System.out.println("space O(n) copy       " + copySpace(16));
        System.out.println("space O(n^2) grid     " + gridSpace(16));
        System.out.println("space O(n) recursion  " + depth(15));
    }
}
```

Go

Go

```go
package main

import "fmt"

func constant(a []int) int { _ = a[0]; return 1 } // O(1)

func logarithmic(n int) int { // O(log n)
    steps := 0
    for n > 1 { n /= 2; steps++ }
    return steps
}

func linear(n int) int { // O(n)
    steps := 0
    for i := 0; i < n; i++ { steps++ }
    return steps
}

func linearithmic(n int) int { // O(n log n)
    steps := 0
    for i := 0; i < n; i++ {
        for m := n; m > 1; m /= 2 { steps++ }
    }
    return steps
}

func quadratic(n int) int { // O(n^2)
    steps := 0
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ { steps++ }
    }
    return steps
}

func exponential(n int) int { // O(2^n)
    if n == 0 { return 1 }
    return exponential(n-1) + exponential(n-1)
}

func factorialTime(n int) int { // O(n!)
    if n == 0 { return 1 }
    total := 0
    for i := 0; i < n; i++ { total += factorialTime(n - 1) }
    return total
}

// ---- space ----
func copySpace(n int) int { return len(make([]int, n)) } // O(n)
func gridSpace(n int) int { // O(n^2)
    total := 0
    for i := 0; i < n; i++ { total += len(make([]int, n)) }
    return total
}
func depth(n int) int { // O(n) stack
    if n == 0 { return 1 }
    return 1 + depth(n-1)
}

func main() {
    fmt.Println("O(1)       ", constant([]int{5, 6, 7}))
    fmt.Println("O(log n)   ", logarithmic(16))
    fmt.Println("O(n)       ", linear(16))
    fmt.Println("O(n log n) ", linearithmic(16))
    fmt.Println("O(n^2)     ", quadratic(16))
    fmt.Println("O(2^n)     ", exponential(10))
    fmt.Println("O(n!)      ", factorialTime(5))
    fmt.Println("space O(n) copy       ", copySpace(16))
    fmt.Println("space O(n^2) grid     ", gridSpace(16))
    fmt.Println("space O(n) recursion  ", depth(15))
}
```

Rust

Rust

```rust
fn constant(a: &[i32]) -> u64 { let _ = a[0]; 1 } // O(1)

fn logarithmic(mut n: u64) -> u64 { // O(log n)
    let mut steps = 0;
    while n > 1 { n /= 2; steps += 1; }
    steps
}

fn linear(n: u64) -> u64 { // O(n)
    let mut steps = 0;
    for _ in 0..n { steps += 1; }
    steps
}

fn linearithmic(n: u64) -> u64 { // O(n log n)
    let mut steps = 0;
    for _ in 0..n {
        let mut m = n;
        while m > 1 { m /= 2; steps += 1; }
    }
    steps
}

fn quadratic(n: u64) -> u64 { // O(n^2)
    let mut steps = 0;
    for _ in 0..n { for _ in 0..n { steps += 1; } }
    steps
}

fn exponential(n: u32) -> u64 { // O(2^n)
    if n == 0 { return 1; }
    exponential(n - 1) + exponential(n - 1)
}

fn factorial_time(n: u32) -> u64 { // O(n!)
    if n == 0 { return 1; }
    (0..n).map(|_| factorial_time(n - 1)).sum()
}

// ---- space ----
fn copy_space(n: usize) -> usize { vec![0; n].len() } // O(n)
fn grid_space(n: usize) -> usize { // O(n^2)
    (0..n).map(|_| vec![0; n].len()).sum()
}
fn depth(n: u32) -> u32 { if n == 0 { 1 } else { 1 + depth(n - 1) } } // O(n) stack

fn main() {
    println!("O(1)        {}", constant(&[5, 6, 7]));
    println!("O(log n)    {}", logarithmic(16));
    println!("O(n)        {}", linear(16));
    println!("O(n log n)  {}", linearithmic(16));
    println!("O(n^2)      {}", quadratic(16));
    println!("O(2^n)      {}", exponential(10));
    println!("O(n!)       {}", factorial_time(5));
    println!("space O(n) copy        {}", copy_space(16));
    println!("space O(n^2) grid      {}", grid_space(16));
    println!("space O(n) recursion   {}", depth(15));
}
```

JavaScript

JavaScript (Node.js)

```javascript
const constant = (a) => { const x = a[0]; return 1; };  // O(1)

function logarithmic(n) {  // O(log n)
  let steps = 0;
  while (n > 1) { n = Math.floor(n / 2); steps++; }
  return steps;
}

function linear(n) {  // O(n)
  let steps = 0;
  for (let i = 0; i < n; i++) steps++;
  return steps;
}

function linearithmic(n) {  // O(n log n)
  let steps = 0;
  for (let i = 0; i < n; i++)
    for (let m = n; m > 1; m = Math.floor(m / 2)) steps++;
  return steps;
}

function quadratic(n) { // O(n^2)
  let steps = 0;
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++) steps++;
  return steps;
}

function exponential(n) {  // O(2^n)
  if (n === 0) return 1;
  return exponential(n - 1) + exponential(n - 1);
}

function factorialTime(n) {  // O(n!)
  if (n === 0) return 1;
  let total = 0;
  for (let i = 0; i < n; i++) total += factorialTime(n - 1);
  return total;
}

// ---- space ----
const copySpace = (n) => new Array(n).fill(0).length;  // O(n)
function gridSpace(n) { // O(n^2)
  let cells = 0;
  for (let i = 0; i < n; i++) cells += new Array(n).fill(0).length;
  return cells;
}
const depth = (n) => (n === 0 ? 1 : 1 + depth(n - 1));  // O(n) stack

console.log("O(1)       ", constant([5, 6, 7]));
console.log("O(log n)   ", logarithmic(16));
console.log("O(n)       ", linear(16));
console.log("O(n log n) ", linearithmic(16));
console.log("O(n^2)     ", quadratic(16));
console.log("O(2^n)     ", exponential(10));
console.log("O(n!)      ", factorialTime(5));
console.log("space O(n) copy       ", copySpace(16));
console.log("space O(n^2) grid     ", gridSpace(16));
console.log("space O(n) recursion  ", depth(15));
```

TypeScript

TypeScript

```typescript
const constant = (a: number[]): number => { const x = a[0]; return 1; };  // O(1)

function logarithmic(n: number): number {  // O(log n)
  let steps = 0;
  while (n > 1) { n = Math.floor(n / 2); steps++; }
  return steps;
}

function linear(n: number): number {  // O(n)
  let steps = 0;
  for (let i = 0; i < n; i++) steps++;
  return steps;
}

function linearithmic(n: number): number {  // O(n log n)
  let steps = 0;
  for (let i = 0; i < n; i++)
    for (let m = n; m > 1; m = Math.floor(m / 2)) steps++;
  return steps;
}

function quadratic(n: number): number { // O(n^2)
  let steps = 0;
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++) steps++;
  return steps;
}

function exponential(n: number): number {  // O(2^n)
  if (n === 0) return 1;
  return exponential(n - 1) + exponential(n - 1);
}

function factorialTime(n: number): number {  // O(n!)
  if (n === 0) return 1;
  let total = 0;
  for (let i = 0; i < n; i++) total += factorialTime(n - 1);
  return total;
}

// ---- space ----
const copySpace = (n: number): number => new Array(n).fill(0).length;  // O(n)
function gridSpace(n: number): number { // O(n^2)
  let cells = 0;
  for (let i = 0; i < n; i++) cells += new Array(n).fill(0).length;
  return cells;
}
const depth = (n: number): number => (n === 0 ? 1 : 1 + depth(n - 1));  // O(n) stack

console.log("O(1)       ", constant([5, 6, 7]));
console.log("O(log n)   ", logarithmic(16));
console.log("O(n)       ", linear(16));
console.log("O(n log n) ", linearithmic(16));
console.log("O(n^2)     ", quadratic(16));
console.log("O(2^n)     ", exponential(10));
console.log("O(n!)      ", factorialTime(5));
console.log("space O(n) copy       ", copySpace(16));
console.log("space O(n^2) grid     ", gridSpace(16));
console.log("space O(n) recursion  ", depth(15));
```

12 Never-Forget Page and Quiz

The one-page memory kit

Class	Scene	Code smell	Doubling n does...
O(1)	Light switch	Direct index, no loop	nothing
O(log n)	Dictionary	Halving the range	adds one step
O(n)	Bookshelf scan	One loop	doubles the work
O(n log n)	Sorting a deck	Divide and merge	slightly more than doubles
O(n^2)	Handshakes	Loop in a loop	quadruples
O(2^n)	Pizza combos	In-or-out recursion, $f(n-1)+f(n-1)$	squares the work
O(n!)	Seating plans	Try every ordering	multiplies by a huge factor

SIX SENTENCES TO MEMORISE

1. Complexity is the *shape of growth*, not seconds.
2. Drop constants and smaller terms; quote the worst case.
3. Halving → **O(log n)**. One pass → **O(n)**. Pairs → **O(n^2)**. Subsets → **O(2^n)**. Orderings → **O(n!)**.
4. Amortized = the average cost over a whole sequence of operations.
5. Space = extra memory, and recursion depth counts.
6. More memory can buy less time, and vice versa.

Quiz: name the time (and space) complexity

1. `for x in items: print(x)`
2. Two nested loops, each running n times.
3. `i = 1; while i < n: i *= 2`
4. `for x in items: for _ in range(10): work()`
5. One loop over n items, followed by a second separate loop over the same n items.
6. A loop over list `a` (size a) with a loop over list `b` (size b) inside it.
7. `def f(n): return 1 if n == 0 else f(n-1) + f(n-1)` (give the time and the space).
8. Copying a list of n items into a new list (give the space).

ANSWERS

1. **O(n)**. 2. **O(n^2)**. 3. **O(log n)**. 4. **O(n)** ($10n$, drop the constant). 5. **O(n)** ($n + n = 2n$, drop the constant). 6. **O(a × b)**.
7. Time **O(2^n)** (two calls per level, n levels). Space is only **O(n)**: the first call finishes before the second starts, so at most n frames are alive at once. 8. Space **O(n)**.

Practice, in your language of choice

1. Write a function that takes n and returns how many halvings it takes to reach 1. Check it against 16 $\rightarrow$ 4 and 1,000,000 $\rightarrow$ 19 (integer halving, rounding down).
2. Time a linear search and a binary search on lists of size 10^3, 10^5 and 10^6. Plot or tabulate how each grows.
3. Reverse an array twice: once by building a new array, once in place. State the space complexity of each.

What Comes Next

Back to the main course: **Lesson 2, the Two-Pointer technique**, which turns many **O(n^2)** brute-force problems into **O(n)** ones with **O(1)** extra space. You now have the vocabulary to see *why* it is faster.